Name: Adnan Khan

Roll No.47

Branch: AI&DS

Class: T12

Subject: ASD&D

Experiment 02

Objective: To containerize an application using Docker

This experiment aligns with LO3:

LO4: Containerizing, orchestrating, and deploying applications with Docker, Compose, Kubernetes, and cloud infrastructure tools.

Theory

Docker is an open-source platform used to develop, package, and deploy applications inside lightweight, portable containers. It ensures that applications run consistently across different environments by bundling the application with all the required libraries, dependencies, and configuration files.

Containerization is the process of packaging an application together with its runtime, libraries, and dependencies into a container. This allows the application to behave identically on a developer's machine, a testing environment, or a production server.

A Docker Image is a read-only template that contains the application code, runtime environment, libraries, and configuration files required to run an application. Images are created using a Dockerfile and serve as the blueprint for creating containers.

A Docker Container is a running instance of a Docker image. It provides an isolated environment in which the application executes independently without affecting the host system

or other applications. Multiple containers can be created from the same Docker image.

Difference Between Docker Image and Docker Container

Docker Image	Docker Container
Read-only template	Running instance of an image
Used to create containers	Used to run applications
Created using a Dockerfile	Created using the “docker run” command
Cannot execute on its own	Executes the application

Advantages of Docker

- Provides application portability across different platforms.
- Offers isolated environments for secure application execution.
- Simplifies dependency and deployment management.
- Consumes fewer system resources than traditional virtual machines.
- Speeds up application development, testing, and deployment.

Docker Installation Steps

1. Download Docker Desktop from the official Docker website.
2. Run the installer and complete the installation process.
3. Restart the computer if required.
4. Launch Docker Desktop and wait until Docker is running.
5. Open Command Prompt or PowerShell and verify the installation using:
   Docker –version
6. Display Docker system information using:
   Docker info
7. Verify the installation by running the Hello World container:
   Docker run hello-world

Commands Executed:

1. Check Docker Version

docker –version

```
C:\Users\15L>docker --version
Docker version 29.2.1, build a5c7197
```

2. Search for Docker Images

docker search ubuntu

This command searches Docker Hub for images related to **Ubuntu**.

```
C:\Users\15L>docker search ubuntu
NAME                            DESCRIPTION                                     STARS     OFFICIAL
ubuntu                          Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid                    Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx                    Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9                    BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/kafka                    Apache Kafka, a distributed event streaming …   62
ubuntu/zookeeper                ZooKeeper maintains configuration informatio…   15
ubuntu/apache2                  Apache, a secure & extensible open-source HT…   104
ubuntu/prometheus               Prometheus is a systems and service monitori…   80
ubuntu/jre                      Chiseled Java runtime based on Ubuntu. Long-…   25
ubuntu/dotnet-aspnet            Chiselled Ubuntu runtime image for ASP.NET a…   26
ubuntu/python                   A chiselled Ubuntu rock with the Python runt…   33
ubuntu/mysql                    MySQL open source fast, stable, multi-thread…   74
ubuntu/postgres                 PostgreSQL is an open source object-relation…   45
ubuntu/redis                    Redis, an open source key-value store. Long-…   26
ubuntu/dotnet-runtime           Chiselled Ubuntu runtime image for .NET apps…   24
ubuntu/dotnet-deps              Chiselled Ubuntu for self-contained .NET & A…   16
ubuntu/grafana                  Grafana, a feature rich metrics dashboard & …   12
ubuntu/memcached                Memcached, in-memory keyvalue store for smal…   7
ubuntu/jdk                      Ubuntu rock for OpenJDK                         0
ubuntu/rabbitmq                 Ubuntu-based RabbitMQ image - an open source…   0
ubuntu/mimir                    Ubuntu ROCK for Mimir, a horizontally scalab…   0
ubuntu/loki                     Grafana Loki, a log aggregation system like …   2
ubuntu/traefik                  Ubuntu ROCK for Traefik, a modern HTTP rever…   3
ubuntu/opentelemetry-collector  This is a rock for the OpenTelemetry Collect…   0
ubuntu/alertmanager             Ubuntu Rock for Alertmanager. It handles ale…   1
```

3. Search with a Filter

docker search ubuntu --filter stars=100

This command displays Ubuntu images that have **100 or more stars** on Docker Hub.

```
C:\Users\15L>docker search ubuntu --filter stars=100
NAME              DESCRIPTION                                     STARS     OFFICIAL
ubuntu            Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid      Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx      Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9      BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/apache2    Apache, a secure & extensible open-source HT…   104
```

4. Download an Ubuntu Image

docker pull ubuntu

This command downloads the latest Ubuntu image from Docker Hub to the local system.

```
C:\Users\15L>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest
```

5. List Downloaded Images

docker images

```
C:\Users\15L>docker images

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   ef54e839ef54       25.9kB         9.52kB   U
nginx:latest         5a88c9c45479        241MB           66MB   U
redis:latest         1c054d54ecd1        204MB         55.3MB   U
ubuntu:latest        3131b4cc82a7        161MB         45.3MB   U
```

6. Run Ubuntu Container

docker run -it ubuntu bash

```
C:\Users\15L>docker run -it ubuntu bash
root@654f898394b6:/#
```

7. Check Ubuntu Version (Inside Container)

cat /etc/os-release

```
C:\Users\15L>docker run -it ubuntu bash
root@654f898394b6:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
```

8. Exit the Container

exit

```
root@654f898394b6:/# exit
exit
```

9. List All Containers

docker ps -a

```
C:\Users\15L>docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS                          PORTS
          NAMES
654f898394b6   ubuntu         "bash"                   About a minute ago   Exited (0) 34 seconds ago
          vigilant_albattani
f7b09cfcb56e   nginx          "/docker-entrypoint.…"   18 hours ago         Created
          practice
e41f9ed60c3c   nginx          "/docker-entrypoint.…"   18 hours ago         Exited (255) 20 minutes ago   0.0.0.0:8080->
80/tcp   mynginx
1000bc0dcc79   ubuntu         "/bin/bash"              19 hours ago         Exited (0) 19 hours ago
          mystifying_wilson
b00193fd8c8f   ubuntu         "/bin/bash"              19 hours ago         Exited (0) 19 hours ago
          jolly_hofstadter
ce280af76907   ubuntu         "/bin/bash"              19 hours ago         Exited (0) 19 hours ago
          sharp_pasteur
04ff3ca987f1   hello-world    "/hello"                 4 months ago         Exited (0) 4 months ago
          focused_agnesi
506fbcc93880   redis:latest   "docker-entrypoint.s…"   4 months ago         Exited (255) 19 hours ago     6379/tcp
          distracted_babbage
d3e8d53b8e82   redis          "docker-entrypoint.s…"   4 months ago         Exited (255) 19 hours ago     6379/tcp
          competent_rosalind
```

10. Start the Stopped Container

First note the **Container ID** from docker ps -a.

docker start <container_id>

```
C:\Users\15L>docker start 04ff3ca987f1
04ff3ca987f1
```

11. Stop the Running Container

docker stop <container_id>

```
C:\Users\15L>docker stop 04ff3ca987f1
04ff3ca987f1
```

12. Inspect a Container

docker inspect <container_id>


```
C:\Users\15L>docker inspect 04ff3ca987f1
[
    {
        "Id": "04ff3ca987f15183d1ee4e0fe422c86e8e26e9319af2cc760921bfd90717d43e",
        "Created": "2026-03-05T09:03:37.015302189Z",
        "Path": "/hello",
        "Args": [],
        "State": {
            "Status": "exited",
            "Running": false,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 0,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-07-21T05:07:31.720720878Z",
            "FinishedAt": "2026-07-21T05:07:31.807310959Z"
        },
        "Image": "sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a",
        "ResolvConfPath": "/var/lib/docker/containers/04ff3ca987f15183d1ee4e0fe422c86e8e26e9319af2cc760921bfd90717d43e/r
```

13. View Running Containers

docker ps

```
C:\Users\15L>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

14. Remove a Container

docker rm <container_id>

```
C:\Users\15L>docker rm 04ff3ca987f1
04ff3ca987f1
```

15. Remove the Ubuntu Image

docker rmi ubuntu

```
C:\Users\15L>docker rmi ubuntu
Untagged: ubuntu:latest
Deleted: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
```

Conclusion

This experiment provided practical exposure to Docker and the fundamentals of containerization. I learned how to install and configure Docker Desktop, verify the installation, and use essential Docker commands for managing images and containers. I gained hands-on experience with commands such as “docker pull”, “docker images”, “docker run”, “docker ps”, “docker stop”, “docker start”, “docker rm”, and “docker rmi”.

I also understood the difference between Docker images and Docker containers, how images are downloaded from Docker Hub, and how containers are created and executed from those images. The experiment demonstrated how Docker provides isolated environments for applications while ensuring consistency across development, testing, and production systems.

Additionally, I learned the advantages of containerization, including application portability, simplified dependency management, faster deployment, efficient resource utilization, and improved scalability. Overall, this experiment strengthened my understanding of Docker’s role in modern software development and DevOps

practices and highlighted its importance in building, deploying, and managing applications efficiently.